# PathMate.ai: An AI-Powered Career Cockpit for Students and Job Seekers

CSE416 Final Report

Dohyeon Lee, Heesang Kim
Department of Computer Science
SUNY Korea / Stony Brook University
Incheon, South Korea
Professor: Yoonseok Yang
June 10, 2026

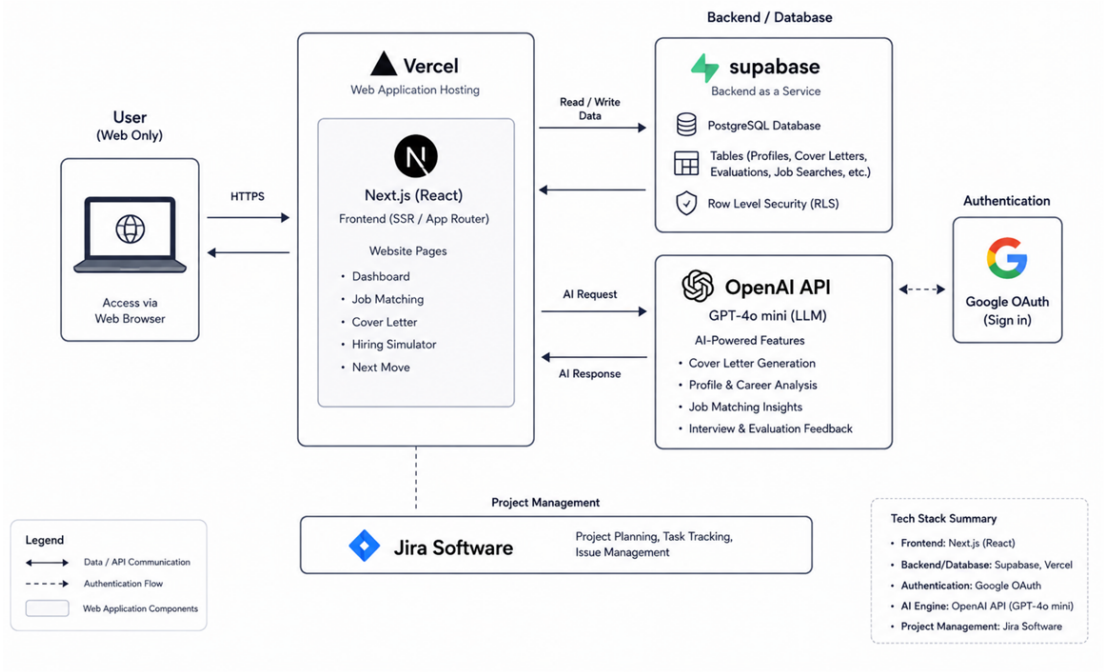



Fig. 1: Architectural overview of PathMate.ai. The system uses a Next.js client, serverless API routes, Supabase authentication and PostgreSQL storage, and OpenAI-powered validation and generation services.

***Abstract*—Modern career preparation is highly fragmented, forcing candidates to use disconnected tools for resume tailoring, job matching, and interview practice. This paper presents PathMate.ai, a unified AI-driven career management ecosystem that centralizes personal competency data to improve the job-seeking pipeline. Developed using Next.js, Supabase with PostgreSQL 16, and OpenAI's GPT-4o-mini, the platform analyzes user profiles, including GPA, technical skill stacks, and project histories, to compute profile-aware fit scores against live-searched recruitment data. It also features real-time input validation, automated cover letter generation, and a profile-aware hiring simulator. The system meets its major functional goals and satisfies a 15-second non-functional latency target for AI-assisted and live-search workflows.**

***Index Terms*—software engineering, career platform, AI assistant, Next.js, Supabase, OpenAI, serverless architecture**

## I. Introduction

### A. Problem Statement

During discussions with fellow students preparing for internships and full-time positions, we noticed that many students relied on multiple separate services for job search, resume writing, and interview preparation. This often made it difficult to keep career-related information organized and consistent. Candidates often interact with separate and uncoordinated applications: one for tracking personal records, another for browsing generic job boards, a third for drafting application essays, and another for conducting mock interviews.

This disconnected workflow creates three major problems. First, traditional job boards often rely on keyword matching and ignore broader academic and project history, such as GPA, course performance, repositories, and certifications. Second,

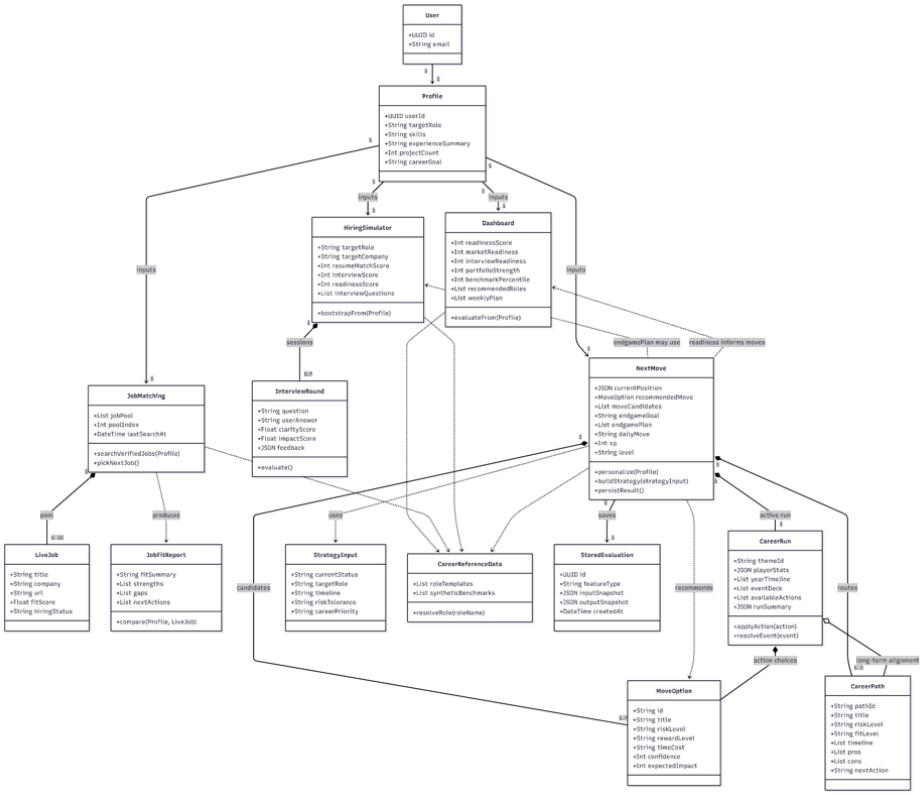



Fig. 2: UML class diagram and relational data model of PathMate.ai.

unstructured profile input can weaken downstream recommendation quality because users may enter vague or unsupported skill descriptions. Third, candidates usually receive little diagnostic feedback after applications, leaving them unaware of specific gaps in their portfolio, technical alignment, or communication quality.

### B. Proposed Solution

PathMate.ai addresses this fragmentation by providing a centralized AI career management platform. Instead of treating resume generation, interview practice, and job matching as separate tasks, the system anchors the user experience around an authenticated profile hub. Our original goal was to combine job matching, cover letter generation, and interview practice into a single platform. As development progressed, we realized that many of these features relied on the same user profile information. This led us to adopt a profile-centered design where academic records, skills, and project experiences could be reused across multiple modules.

## II. Requirements Summary

Table I summarizes the major functional and non-functional requirements used to guide the implementation. These requirements focus on what the system should provide to the user rather than internal implementation details.

## III. System Architecture and Design

### A. Architectural Overview

PathMate.ai uses a full-stack serverless architecture built on the Next.js framework. The system avoids a separate dedicated backend server by using Next.js API routes, deployed as serverless functions on Vercel. This design reduces deployment friction and keeps the frontend and backend logic within one project structure.

The architecture contains four key interaction layers. First, the React frontend sends HTTP requests to serverless endpoints and includes the user's active session token in the `Authorization` header. Second, the browser uses the Supabase anonymous key for UI-driven operations, restricted by PostgreSQL Row-Level Security. Third, the serverless layer calls the OpenAI SDK using private environment variables that are never exposed to the client. Fourth, backend routes use the Supabase service role key only for privileged multi-table operations and controlled server-side workflows.

### B. Core Component Design

The system's data model is centered on the authenticated user profile. The profile entity connects to generated cover letters, interview sessions, dashboard insight records, job matching results, and Next Move scenario data. This organization supports a profile-centric architecture in which user data becomes the shared foundation for multiple career preparation features.

TABLE I: Requirements Summary

| ID | Type | Requirement | Implemented By |
|---|---|---|---|
| FR-1 | Functional | Users shall authenticate using Google OAuth before accessing protected career features. | Supabase Auth, Google OAuth |
| FR-2 | Functional | Users shall create and update structured career profiles containing academic, skill, project, and career preference data. | Settings dashboard, profile API |
| FR-3 | Functional | The system shall validate profile inputs before saving them to reduce vague or corrupted data. | `profile-validation.ts`, `/api/profile-validate` |
| FR-4 | Functional | Users shall generate profile-aware cover letters from resume text and job descriptions. | Resume Lab, `/api/cover-letter` |
| FR-5 | Functional | Users shall practice interviews through personalized questions and receive answer evaluations. | Hiring Simulator APIs |
| FR-6 | Functional | Users shall receive job recommendations based on their profile and live recruitment data. | `/api/matching-real-jobs` |
| NFR-1 | Performance | AI-assisted and live-search workflows should complete within 15 seconds under tested conditions. | Optimized serverless API routes |
| NFR-2 | Security | User data should be isolated from unauthorized access. | Supabase RLS, server-side service role separation |

### C. Profile Validation Flow

To prevent low-quality profile data from weakening downstream AI outputs, PathMate.ai uses a two-tier validation pipeline whenever users modify profile fields in the Settings hub. The first tier applies deterministic heuristic checks using utilities in `lib/profile-validation.ts`. If fields match keyboard-mashing patterns or contain overly generic placeholders such as "developer" in the target role field, the backend rejects the update with an HTTP 400 response.

The second tier performs AI semantic inspection. Once heuristic checks pass, selected fields are sent to `POST /api/profile-validate`. The model evaluates whether the submitted profile fields are semantically compatible, such as whether the stated technical skills match the user's major or target role. If OpenAI fails due to a parsing or network error, the route falls back to the successful heuristic result and permits the database update with a skipped AI-review flag.

## IV. Implementation

### A. Career Dashboard and Analytics

The Dashboard acts as the visual center of the platform. It uses a deterministic heuristic evaluation route, `GET /api/dashboard-insight`, rather than making runtime AI calls. This reduces latency and makes the dashboard output more predictable.

The server computes a multidimensional readiness matrix across four axes: *profileCompleteness*, *marketReadiness*, *interviewReadiness*, and *portfolioStrength*. Profile completeness measures whether required profile fields have been completed. Market readiness considers internships, certificates, and regional career constraints. Interview readiness uses the number and quality of completed simulator sessions. Portfolio strength checks for professional links such as GitHub repositories or public portfolios.

During development, we created a small synthetic dataset of approximately 500 profiles to provide a reference baseline for dashboard feedback. The dataset was manually generated using combinations of GPA ranges, project experience levels, certifications, and internship histories. While this dataset is not intended to represent real-world statistics, it provides a simple benchmark for relative readiness scoring.

### B. Resume Lab

The Resume Lab generates personalized cover letters through `POST /api/cover-letter`. Users provide plain-text resume fragments and job descriptions, then select a desired tone such as professional, confident, concise, or realistic.

The system injects validated user profile data into the prompt context to improve relevance. To reduce unsupported claims, the prompt policy discourages hallucination and embellishment. In realistic tone mode, the model is instructed not to invent experience that is not present in the user profile or resume input. If an output attempts to rely on missing evidence, the UI can warn the user through a Missing Evidence Flag.

### C. Hiring Simulator

The Hiring Simulator uses two main routes. The first route, `POST /api/personalized-hiring-simulator`, validates the target company and role, reads the user profile, and considers recent generated cover letters to construct interview context. It then generates six interview questions across behavioral and technical categories.

The second route, `POST /api/evaluate/hiring-manager`, evaluates submitted interview answers. The client provides lightweight typing feedback through React `useMemo` hooks, while the backend performs the final answer assessment. If the answer is too short or consists of gibberish, the API rejects it with an HTTP 400 response. If the AI service is unavailable, the route returns HTTP 503 instead of producing a fake evaluation.

### D. Next Move

The Next Move module provides a gamified career roadmap interface. In the current client implementation, it renders a board-style simulation using static data from `mockNextMoveData.ts`. Users choose a career path and

TABLE II: Requirements Traceability Matrix

| ID | Requirement | Implementation | Verification |
|---|---|---|---|
| FR-1 | Authenticate users with Google OAuth. | Supabase Auth and protected routes. | Login/logout test |
| FR-2 | Save structured user profile data. | Settings dashboard and Supabase profile table. | Save/reload test |
| FR-3 | Validate profile fields before saving. | Heuristic validator and AI validation endpoint. | Invalid input rejection test |
| FR-4 | Generate profile-aware cover letters. | Resume Lab and cover letter endpoint. | Prompt/output test |
| FR-5 | Generate and evaluate mock interviews. | Hiring Simulator and evaluation route. | Session/evaluation test |
| FR-6 | Recommend jobs from live recruitment sources. | Matching engine and link validation helper. | Job matching test |
| NFR-1 | Complete AI-assisted workflows under 15 seconds. | Serverless route optimization and prompt control. | Latency test |
| NFR-2 | Protect user data from unauthorized access. | RLS, session tokens, service role separation. | Auth ownership test |

progress through annual cycles, observing how decisions affect salary, skill, burnout, network, and reputation.

There is currently an integration gap between the frontend simulation and the backend endpoints. The frontend relies on static mock data for stable demo performance, while the backend routes `/api/personalized-next-move`, `/api/evaluate/career-chess`, and `/api/evaluate/career-readiness` were implemented as support for future profile-driven roadmap generation.

### E. Job Matching Engine

The Job Matching Engine, implemented through `GET /api/matching-real-jobs`, recommends jobs using profile-aware matching and live recruitment search. The route first checks whether the user's profile satisfies minimum completeness requirements. If the profile is incomplete, the system returns HTTP 422 and blocks matching.

The engine restricts results to selected recruitment domains in South Korea, including `jobkorea.co.kr`, `wanted.co.kr`, `rocketpunch.com`, `catch.co.kr`, `incruit.com`, and `peoplenjob.com`. Before returning a result, the backend attempts link validation through `verifyLiveJobPosting()`. Closed or unreachable postings are removed when detected, reducing the chance of sending users to unavailable jobs.

## V. Testing and Verification

### A. Requirements Traceability

Table II shows how the implemented modules map to the major requirements. This traceability view confirms that each core requirement has a corresponding implementation path and verification method.

### B. Functional and Performance Tests

The critical performance target required AI-assisted queries and live background computations to complete within a 15-second service-level threshold. The longest tested route occurred during a cache miss in the job matching workflow because the server had to coordinate keyword queries, live web search, and destination link checks. Even under this heavier path, the measured response time was 8.40 seconds, remaining below the 15-second requirement. We also observed that vague profile descriptions frequently produced poor recommendations, which motivated the addition of profile validation before matching requests were processed.We also observed that vague profile descriptions frequently produced poor recommendations, which motivated the addition of profile validation before matching requests were processed.

TABLE III: Test Case Summary

| ID | Test Case | Result |
|---|---|---|
| T1 | Login with Google OAuth and reach dashboard. | Pass |
| T2 | Submit vague target role such as "developer". | HTTP 400, Pass |
| T3 | Submit valid structured profile data. | Saved, Pass |
| T4 | Generate cover letter from resume and job description. | Generated, Pass |
| T5 | Submit short or gibberish interview answer. | HTTP 400, Pass |
| T6 | Run job matching with incomplete profile. | HTTP 422, Pass |
| T7 | Run maximum-load matching route. | 8.40s, Pass |

## VI. Discussion

### A. Current Limitations

The current beta version of PathMate.ai has several limitations. First, Resume Lab currently processes plain-text resume fields only. It does not yet include native PDF parsing, so users must manually copy and paste resume content. Second, although the database stores a history of generated cover letters, the API lacks granular version-control actions such as deletion, rollback, and side-by-side comparison. Third, the Dashboard includes a separate readiness computation file, `lib/dashboard-readiness.ts`, but the production dashboard insight route still relies partly on internal component utilities.

### B. Future Enhancements

Two immediate improvements are planned. The first is client-side PDF document processing, which would extract structured resume content without increasing backend load. The second is a Git-style visual comparison system for Resume Lab, allowing users to inspect keyword additions, tone changes, and edits across generated versions.

## VII. Conclusion

Through this project, we learned that many career-preparation features can be built around a shared user profile rather than as isolated tools. Developing PathMate.ai also highlighted practical challenges such as profile validation, AI response reliability, and external job-data integration. Although several features remain incomplete, the current system provides a working foundation that can be extended in future iterations.

## VIII. Contributions

**Dohyeon Lee** served as frontend engineer and UI designer. He drafted the initial product concept and scope, implemented the frontend interfaces using Next.js and Tailwind CSS 4.0, developed responsive layout shells for the Career Dashboard, Resume Lab, and Hiring Simulator, and managed frontend testing and UI polish through GitHub Issues.

**Heesang Kim** served as backend architect and system designer. He designed the database schemas, entity relationships, and system data dictionary, implemented the OpenAI orchestration filters and Supabase access layer, and managed the unified GitHub repository, database infrastructure, and deployment pipeline on Vercel.